SCE 672 | TERM PROJECT ASSESSMENT 1

RoboMaster EP: Provisioning, Teleop, ROS 2

Keyboard teleoperation, telemetry, and middleware integration

Ameen Ur Rehman (202527430) | Prof. Aamer Iqbal Bhatti | KFUPM, 2026

Problem 1: Host environment and middleware

(i)–(iv) Provisioning snapshots

Captured from the lab host with the commands shown (raw file: `logs/system_diagnostics.txt`). Kernel, Ubuntu 24.04 LTS, Python 3.12, ROS 2 Jazzy, Gazebo Harmonic (Gazebo Sim 8.x) and NumPy 1.26.4 inside `~/rm_venv` are all in place; `robomaster` SDK (community fork) and `robomaster_ros` were built with `colcon build --symlink-install`.

```
$ uname -srmo
Linux 7.0.0-38-generic x86_64 GNU/Linux

$ lsb_release -a
Distributor ID:        Ubuntu
Description:        Ubuntu 24.04.5 LTS
Release:        24.04
Codename:        noble

$ python3 --version
Python 3.12.3

$ echo $ROS_DISTRO ; ros2 --help | head -1
jazzy
ros-jazzy-desktop-full 0.11.0-1noble.20261006.041807
ros-jazzy-ros-gz 1.0.24-1noble.20261006.041805

$ source ~/rm_setup.sh && python -c "import numpy ..."
/home/ameen/rm_venv/bin/python
numpy 1.26.4
numpy                                1.26.4
numpy-quaternion                     2024.0.13
PyYAML                               6.0.1
rm_libmedia_codec                    0.0.1
robomaster                           0.1.2.0

$ gz sim --versions
Gazebo Sim, version 8.15.0
Copyright (C) 2018 Open Source Robotics Foundation.

$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2       233G   30G  192G  14% /
```

(v) ABI incompatibility analysis

Why `pip install -U numpy` breaks ROS 2 nodes. NumPy 2.0 changed the C-API and ABI. Struct layouts exposed to extensions (for example `PyArray_Descr`, whose `elsize` and `alignment` fields were widened and moved, and removed legacy members) and the function table behind `PyArray_API` were reorganised. A compiled extension does not call NumPy by name. It indexes into that table and reads those structs at *fixed offsets baked in at build*

time. The Ubuntu/ROS binaries (`cv_bridge`'s Boost.Python module, `rclpy` message converters, OpenCV's `cv2`) are compiled against NumPy 1.26. If NumPy 2.x is what gets imported at run time, those offsets point at the wrong slots and fields. The outcome is one of two failure modes: NumPy's guard raises `ImportError: A module that was compiled using NumPy 1.x cannot be run in NumPy 2.x`, or, where the mismatch goes unchecked, the extension dereferences a wrong pointer and the interpreter dies with a segmentation fault. Python-level code is unaffected, which is why the failure shows up only at the C boundary (image conversion, point-cloud and array message handling).

Why a plain `-U` triggers it. With a venv created using `--system-site-packages`, an unconstrained upgrade installs NumPy 2.x into the venv's own `site-packages`. That directory comes first on `sys.path`, so it *shadows* the apt-provided 1.26.4 that the ROS binaries were built for. The same shadowing happens if a user-level copy sits in `~/.local`.

How the setup isolates and resolves it.

- `python -m venv --system-site-packages`: the interpreter can see ROS's compiled bindings (`rclpy`, `cv_bridge`) while pip installs stay out of the system Python, which PEP 668 protects.
- `"numpy<2"`: the explicit pin forbids pip from ever resolving to 2.x (the RoboMaster SDK and `numpy-quaternion` would otherwise pull the latest).
- `rm_setup.sh`: sources ROS and the workspace overlay, activates the venv, prepends the venv `site-packages` to `PYTHONPATH`, and exports `PYTHONNOUSERSITE=1` so `~/.local` packages cannot shadow anything. Every shell therefore resolves the same, ABI-consistent NumPy.

Observation on this host. `import numpy` resolves to `/usr/lib/python3/dist-packages` (1.26.4), because the pin was already satisfied by the system copy and pip therefore installed no second one. The effect is the same: one NumPy, the one ROS was compiled against.

Problem 2: SDK teleoperation and telemetry

(i) Implementation: `ep_teleop_sdk.py`

A single-file script, importing only `robomaster` (no ROS 2 imports).

- **Keyboard:** `curses`, non-blocking. W/S $\rightarrow v_x$, A/D $\rightarrow v_y$, Q/E $\rightarrow \omega_z$. Terminals report presses and auto-repeat but never key release, so each key carries a hold window (0.55 s for the first press to cover the OS typematic delay, 0.15 s once repeating). When repeats stop the key counts as released and its contribution drops out. Keys can combine (e.g. forward plus strafe).
- **Dispatch:** commands go to `ep_robot.chassis.drive_speed()` at 20 Hz.
- **Telemetry:** `sub_battery_info`, `sub_attitude` and `sub_esc` are subscribed at 10 Hz. A logger writes one CSV row per 100 ms: battery state of charge, yaw/pitch/roll, four wheel speeds (rpm), the arrival time of the newest ESC packet, and the commanded twist.
- **Failsafe:** Space zeroes $v_x = v_y = \omega_z = 0$ immediately (sent in the same loop iteration, not on the next 20 Hz tick). Releasing keys stops the chassis through the hold window. The `finally` block sends stop commands and unsubscribes on any exit, including exceptions.

SDK sign convention: x forward, y to the *right*, ω_z *clockwise*. This is the opposite of ROS REP-103 (y left, ω_z counter-clockwise); the driver handles the conversion for the ROS runs.

Logic check without hardware. The control path was exercised against a stub SDK in a pseudo-terminal: forward hold produced $v_x = 0.3$ m/s, forward plus strafe combined, both

dropped to zero 0.45 s after the last keypress, yaw produced $45°$/s, and Space zeroed all commands within 6 ms. The CSV logger wrote 10 rows/s. This is a software test only and is *not* the hardware result.

(ii) Wired RNDIS run (30 s)

Controller switch on Direct/USB, Micro-USB data cable, `python ep_teleop_sdk.py --conn rndis --duration 30 --out logs/rndis_run.csv`. **Result.** Complete 30 s run: 299 rows over 29.9 s (10 Hz) with battery, attitude and all four wheel speeds logged and no missing values; the newest ESC packet arrived 0.04 s before the file ended, so the link stayed up for the whole run. Forward and lateral commands were used ($|v_x| = |v_y| = 0.3$ m/s), yaw was not. The telemetry arrival interval was 102.3 ± 8.8 ms (min 98.3, max 202.5, 95th percentile 107.0, $n = 291$); the single 200 ms gap is one late packet. (An earlier wired attempt was cut to 9.6 s by a USB enumeration failure and was replaced by this run.) **[TODO: attach terminal snapshot of the live telemetry display]**

(iii) Wireless AP run (30 s)

Switch on AP mode, laptop joined to the robot's access point (network profile `RMEP-21b442`), `python ep_teleop_sdk.py --conn ap --duration 30 --out logs/ap_run.csv`. **Result.** 299 rows over 29.9 s (10 Hz); battery, attitude and wheel speeds were logged and all three motions were used (forward and lateral at 0.3 m/s, yaw at $45°$/s). The ESC telemetry arrival interval was 120.2 ± 52.3 ms (min 17.3, max 319.6, 95th percentile 222.7, $n = 247$). **[TODO: attach terminal snapshot of the live telemetry display]**

(iv) Comparative latency analysis

Ping: `scripts/ping_bench.sh rndis` and `ap` (100 packets each). Plots by `scripts/analyze_telemetry.py`. The telemetry interval is $\Delta t = t_k - t_{k-1}$ over ESC packet arrival times (network-level), next to the logger loop interval (host scheduling only).

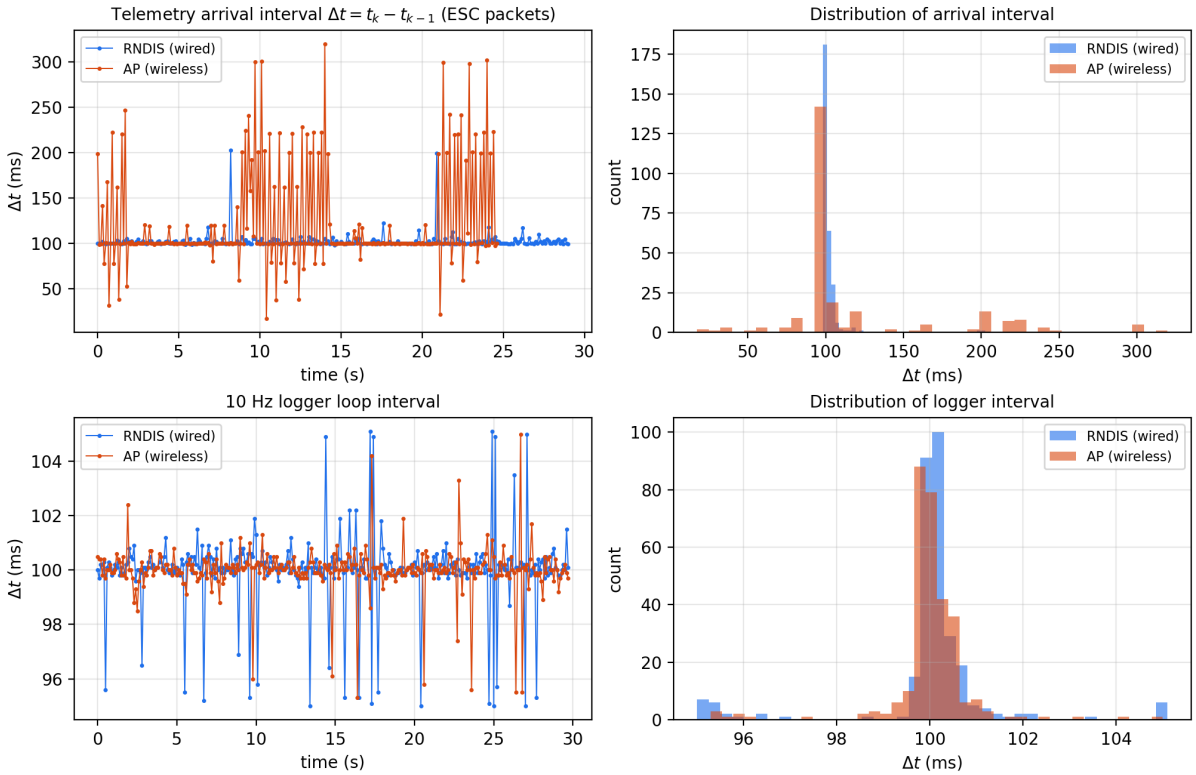


Figure 1: Telemetry sampling interval Δt for wired RNDIS and wireless AP.

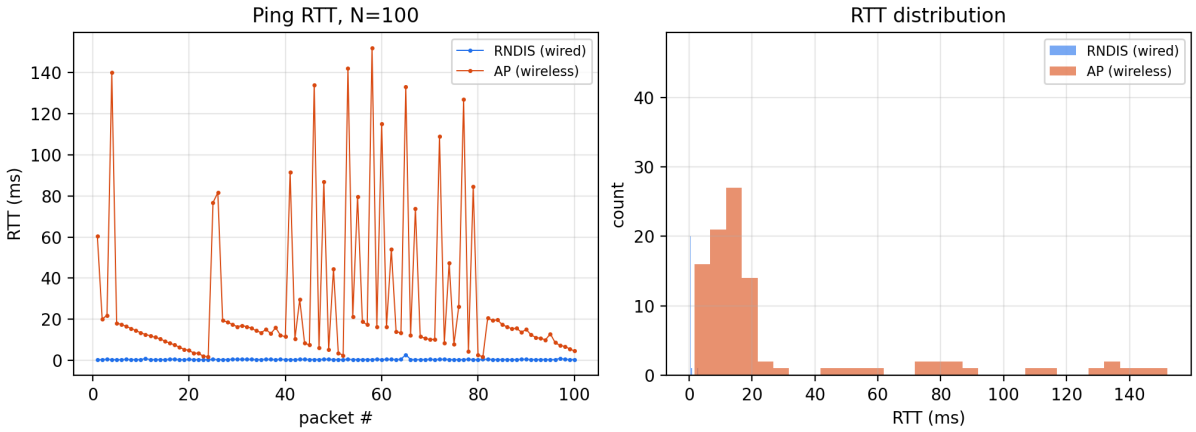


Figure 2: Ping round-trip time, $N = 100$.

	Wired RNDIS	Wireless AP
Ping loss ($N = 100$)	0 %	0 %
RTT mean (ms)	0.451	28.3
RTT std (ms)	0.252	36.7
Jitter, mean $\lvert \mathrm{RTT}_i - \mathrm{RTT}_{i-1} \rvert$ (ms)	0.137	32.1
RTT min / 95th pct / max (ms)	0.28 / 0.56 / 2.75	1.68 / 127.3 / 152.0
Telemetry Δt mean $\pm$ std (ms)	102.3 ± 8.8 (n=291)	120.2 ± 52.3 (n=247)
Telemetry Δt min / max (ms)	98.3 / 202.5	17.3 / 319.6

Measured latency statistics. Both telemetry runs are complete 30 s logs.

Discussion. Wireless AP responsiveness is clearly worse than wired, even though neither link

lost any ping.

- **Latency:** mean RTT rises from 0.45 ms to 28.3 ms, about 63 times. The wired link never exceeded 2.8 ms, while the AP link reached 152 ms.
- **Jitter:** the AP RTT has a standard deviation of 36.7 ms (larger than its own mean), against 0.25 ms wired, and a mean successive difference of 32.1 ms against 0.14 ms. The distribution is heavy-tailed: the 95th percentile is 127.3 ms against 0.56 ms.
- **Telemetry intervals:** wired packets arrive almost exactly every 100 ms (102.3 ± 8.8 ms, with one 202 ms outlier). Over AP the stream is bursty (Figure 1): gaps of about 200 to 320 ms are followed by very short intervals, down to 17 ms, as delayed packets are delivered together. The mean stays near the nominal period (120 ms) while the standard deviation is about six times larger. The logger saw 248 distinct ESC packets in 299 ticks, so stale values were repeated during the delays.
- **Command lag:** a command takes one-way delay plus up to 50 ms for the 20 Hz dispatch. With RTT spikes up to 150 ms over AP, a key release or Space press can reach the robot about 0.15 s late, which at 0.3 m/s is roughly 4.5 cm of extra travel.

Further points:

- **Responsiveness:** wired USB is a point-to-point link with no contention, so RTT is low and tight. A 2.4/5 GHz AP adds medium-access contention, retransmissions and power-save wake-ups, which raises the mean and, more importantly, the tail.
- **Command lag:** a command reaches the robot after one-way delay plus the 20 Hz dispatch period (up to 50 ms) and the 100 ms telemetry period bounds how stale the feedback is. Over AP the one-way delay variance can exceed the dispatch period, so lag becomes visibly irregular.
- **Jitter:** appears as spread in Δt: a 10 Hz stream should give $\Delta t \approx 100$ ms; late packets followed by bursts give a heavy right tail and a compensating short interval.
- **Safety link:** a missed stop packet over AP is the practical risk, which is why the Space key and release timeout matter more on the wireless link.

Problem 3: Wireless teleoperation and telemetry in ROS 2

(i) Driver over hotspot

```
source ~/rm_setup.sh
ros2 launch robomaster_ros ep.launch conn_type:=ap
```

Verification that the node initialises, advertises `/cmd_vel`, and publishes `/odom` and `/joint_states`: The driver log reports `Try to connect via ap to robot with sn None` followed by `Connected`. Running nodes: `/robomaster`, `/robot_state_publisher`, `/joint_state_publisher`. `/cmd_vel` has one subscriber (`/robomaster`), and `/odom`, `/joint_states`, `/imu`, `/battery`, `/tf` are advertised. Measured rates (8 s windows, `logs/ap_ros_verification.txt`):

- `/odom`: 9.8 Hz average, with intervals between 4 ms and 226 ms (std 46 to 51 ms).
- `/joint_states`: 10.2 Hz average, intervals between 19 ms and 101 ms (std 12 to 14 ms).

Both topics average the configured 10 Hz. The `/odom` intervals spread much wider (4 to 226 ms), consistent with odometry packets arriving over Wi-Fi in bursts as in Problem 2.

(ii) Keyboard pipeline

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
    --ros-args -r /cmd_vel:=/cmd_vel
```

The run lasted about 289 s with 81 `/cmd_vel` messages (maximum $|v_x| = 0.55$ m/s, $|\omega_z| = 1.0$ rad/s), with roughly 21 s of forward driving and 40 s of rotation; the net heading change is $+1447°$ (about four turns), from repeated in-place turns at the corners of the path. **No lateral strafing was commanded** (max $|v_y| = 0$ in the bag), so the strafing requirement of this step is not yet met. **[TODO: short repeat run with Shift+J / Shift+L strafing legs]**

(iii) Telemetry acquisition

```
ros2 bag record -o logs/bag_ap /odom /joint_states /cmd_vel
```

Result (`logs/ap_bag_info.txt`): 289.5 s, 5804 messages, MCAP storage: `/odom` 2827 (9.8 Hz), `/joint_states` 2896 (10.0 Hz), `/cmd_vel` 81. The largest gap between consecutive `/odom` messages is 0.26 s and only 3 gaps exceed 0.25 s, so recording was continuous with no network stall.

(iv) State estimation and odometry trajectory

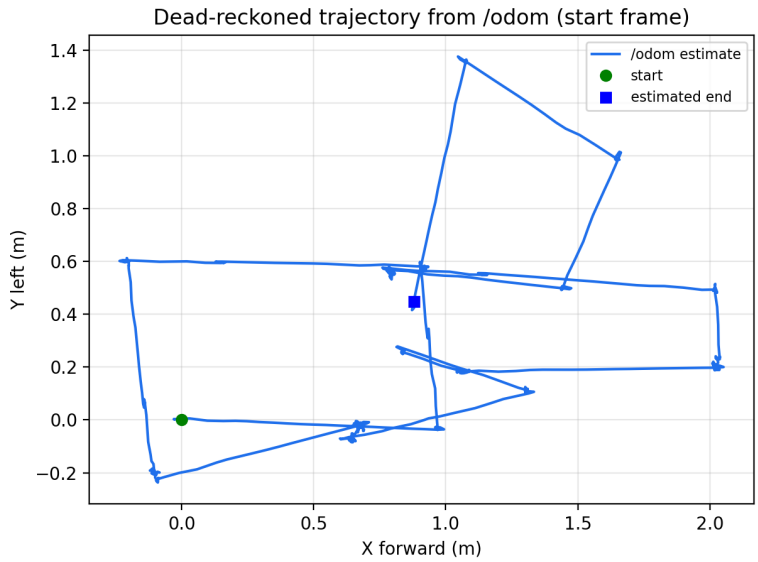


Figure 3: Estimated 2-D trajectory from `/odom`.

Endpoint comparison. Expressed in the start pose frame (x forward, y left), the estimated end is $(X, Y) = (0.881,\ 0.447)$ m after an integrated path length of 14.86 m. **[TODO: tape-measured true end position (same frame) and the resulting error in m and as % of path length: not measured for this run]**

Sources of drift.

- **Dead-reckoning integration:** pose is the running integral $\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{R}(\psi_k)\mathbf{v}_k \Delta t$ of measured wheel velocities. Each step's error is never corrected, so position error grows without

bound, and heading error is the worst contributor because it rotates every later displacement: a heading error $\delta\psi$ yields a lateral position error of about $d\,\delta\psi$ after travelling distance d.

- **Mecanum roller slip:** the lateral and yaw motions rely on rollers sliding at 45°, so wheel rotation is only loosely tied to ground displacement. Strafing and rotating slip much more than driving straight, and the effective wheel radius changes with the contact point on the rollers. Wheel encoders measure the wheel, not the ground, so slip is invisible to odometry.
- **Model and discretisation errors:** the nominal R, l_x, l_y differ from the real values (tyre wear, load), and 10 Hz sampling with first-order integration ignores motion within a step during turns. Accelerations cause transient slip too.
- **Sensing noise:** gyro bias and encoder quantisation add a slowly growing error, and any late or dropped `/odom` packet over Wi-Fi coarsens the integration step.

Problem 4: Theory and diagnostics

(i) Network topology characterisation

Qualitative rows follow from the topologies; measured rows come from the runs above.

Metric	Wired USB (`rndis`)	Direct hotspot (`ap`)
Host interface name	`enx52a542fd448f`	`wlp1s0`
Host IP and netmask	192.168.42.3/24	192.168.2.32/24
Robot endpoint IP	192.168.42.2	192.168.2.1
Round-trip latency (avg ± std, $N = 100$)	0.45 ± 0.25 ms (jitter 0.14 ms)	28.3 ± 36.7 ms (jitter 32.1 ms)
Host internet retained?	Yes (observed: the Wi-Fi stayed connected while the USB link was up).	No (observed: the AP profile has no gateway and the only Wi-Fi radio is tied to the robot).
Multi-robot scalability	Poor. One cable and one USB interface per robot, and every robot uses the same 192.168.42.2 subnet, so they collide on one host without extra isolation.	Poor. A host Wi-Fi adapter joins one AP at a time and every robot AP uses 192.168.2.1, so one adapter serves one robot.

Table 1: Physical connection topology comparison.

(ii) Mecanum kinematic derivation

Setup. Body frame: x forward, y left, ω_z counter-clockwise (as in Figure 1 of the assignment). Wheel i has radius R and angular velocity ω_i (positive drives the vehicle forward) and sits at (x_i, y_i) with $x_i = \pm l_x$, $y_i = \pm l_y$:

$$\text{FL} = (+l_x, +l_y), \quad \text{FR} = (+l_x, -l_y), \quad \text{RL} = (-l_x, +l_y), \quad \text{RR} = (-l_x, -l_y).$$

Rollers sit at angle $\gamma_i = \pm 45°$ to the wheel's driving direction (the standard RoboMaster layout: FL and RR have one roller handedness, FR and RL the other). The figure does not draw rollers, so this is the assumed layout.

Step 1: velocity of the contact point. For a rigid body twist $\mathbf{v}_b = [v_x, v_y, \omega_z]^T$ the velocity of the point at (x_i, y_i) is

$$\mathbf{v}_i = \begin{bmatrix} v_x - \omega_z y_i \\ v_y + \omega_z x_i \end{bmatrix}.$$

Step 2: wheel and roller constraint. The contact point moves with the rim speed $u_i = R\omega_i$ along the wheel's driving direction $\hat{\mathbf{x}}$, plus a free sliding speed s_i along the roller axis $\mathbf{r}_i = (\cos\gamma_i, \sin\gamma_i)$:

$$\mathbf{v}_i = u_i\hat{\mathbf{x}} + s_i\mathbf{r}_i.$$

The y component gives $s_i = v_{iy}/\sin\gamma_i$. Substituting into the x component:

$$R\omega_i = u_i = v_{ix} - v_{iy}\cot\gamma_i, \qquad \sigma_i := \cot\gamma_i = \pm 1.$$

Step 3: insert the contact velocity.

$$R\omega_i = v_x - \sigma_i v_y - \omega_z\,(y_i + \sigma_i x_i).$$

With $\sigma = (+1, -1, -1, +1)$ for (FL, FR, RL, RR), every term $y_i + \sigma_i x_i$ equals $\pm(l_x + l_y)$. Define $k = l_x + l_y$:

$$\omega_{fl} = \tfrac{1}{R}\,(v_x - v_y - k\,\omega_z), \quad \omega_{fr} = \tfrac{1}{R}\,(v_x + v_y + k\,\omega_z),$$
$$\omega_{rl} = \tfrac{1}{R}\,(v_x + v_y - k\,\omega_z), \quad \omega_{rr} = \tfrac{1}{R}\,(v_x - v_y + k\,\omega_z).$$

Inverse kinematics.

$$\mathbf{J}_{IK} = \frac{1}{R}\begin{bmatrix} 1 & -1 & -k \\ 1 & 1 & k \\ 1 & 1 & -k \\ 1 & -1 & k \end{bmatrix}$$

Forward kinematics. $\mathbf{J}_{IK}$ is 4×3 with full column rank, so the least-squares inverse is the pseudo-inverse $\mathbf{J}_{FK} = (\mathbf{J}_{IK}^T\mathbf{J}_{IK})^{-1}\mathbf{J}_{IK}^T$. Its three columns $\mathbf{c}_1 = (1,1,1,1)$, $\mathbf{c}_2 = (-1,1,1,-1)$, $\mathbf{c}_3 = (-k,k,-k,k)$ (up to the factor $1/R$) are mutually orthogonal: $\mathbf{c}_1 \cdot \mathbf{c}_2 = 0$, $\mathbf{c}_1 \cdot \mathbf{c}_3 = 0$, $\mathbf{c}_2 \cdot \mathbf{c}_3 = k + k - k - k = 0$. Hence

$$\mathbf{J}_{IK}^T\mathbf{J}_{IK} = \frac{1}{R^2}\,\mathrm{diag}(4,\,4,\,4k^2),$$

which gives

$$\mathbf{J}_{FK} = \frac{R}{4}\begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & 1 & -1 \\ -\frac{1}{k} & \frac{1}{k} & -\frac{1}{k} & \frac{1}{k} \end{bmatrix}, \qquad \begin{aligned} v_x &= \tfrac{R}{4}(\omega_{fl} + \omega_{fr} + \omega_{rl} + \omega_{rr}) \\ v_y &= \tfrac{R}{4}(-\omega_{fl} + \omega_{fr} + \omega_{rl} - \omega_{rr}) \\ \omega_z &= \tfrac{R}{4k}(-\omega_{fl} + \omega_{fr} - \omega_{rl} + \omega_{rr}). \end{aligned}$$

Check. $\mathbf{J}_{FK}\mathbf{J}_{IK} = \mathbf{I}_3$. For example, the $(1,1)$ entry is $\frac{R}{4} \cdot \frac{1}{R}(1+1+1+1) = 1$ and the $(3,3)$ entry is $\frac{R}{4k} \cdot \frac{1}{R}(k+k+k+k) = 1$. $\mathbf{J}_{IK}\mathbf{J}_{FK} \neq \mathbf{I}_4$: four wheels span only a 3-D twist space, and the left-over direction $(1,-1,-1,1)$ is a wheel-speed combination that produces no body motion, which is why wheels fighting each other show up as slip.

Sanity checks. Forward: all wheels $+v_x/R$. Strafe left ($v_y > 0$): FL and RR spin backward, FR and RL forward. Rotation ($\omega_z > 0$, CCW): left wheels backward, right wheels forward. All match the known mecanum behaviour.

Numbers. Nominal RoboMaster EP values (confirm against the datasheet before quoting): $R = 0.05\,\mathrm{m}$, $l_x = l_y = 0.10\,\mathrm{m}$, so $k = 0.20\,\mathrm{m}$. Conversion to the SDK convention (y right, ω_z clockwise): $v_y^{\mathrm{SDK}} = -v_y$, $\omega_z^{\mathrm{SDK}} = -\omega_z$.

(iii) ROS 2 coordinate transform inspection

`ros2 run tf2_tools view_frames` was run with the driver up.

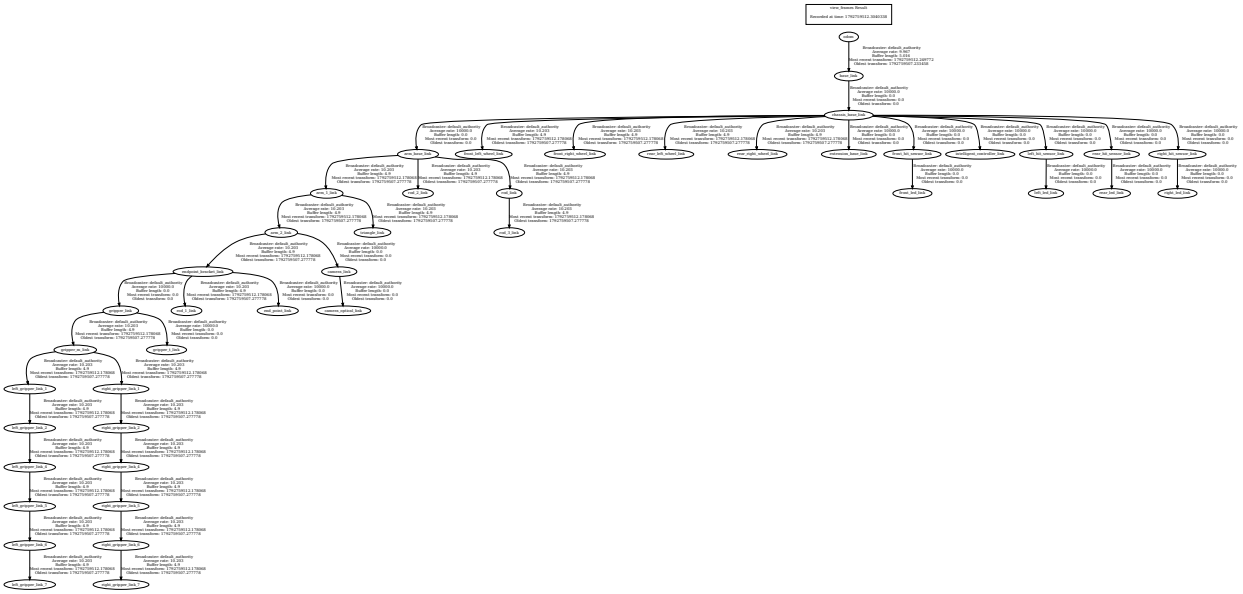


Figure 4: TF tree generated by `tf2_tools view_frames` (vector PDF, zoom in for detail).

Parent and child links, read from the generated tree (`figures/frames.pdf`) and `ros2 topic info /tf -v`, which lists exactly two publishers of `/tf`:

- `odom` → `base_link`: the only dynamic link at the top of the tree, published at about 10 Hz by the driver node `/robomaster` (its chassis module, from the dead-reckoned pose). `odom` is the root of the tree.
- `base_link` → `chassis_base_link`: a fixed (static) link from the robot description, published by `/robot_state_publisher`.
- `chassis_base_link` → `arm_base_link`: a fixed (static) link (`arm_base_joint`, offset $(0,\ 0.0010,\ 0.0906)$ m), also published by `/robot_state_publisher`.

Beyond these, `/robot_state_publisher` also publishes the moving links, driven by `/joint_states`: the four wheel links (children of `chassis_base_link`) and the arm and gripper chain (`arm_base_link` → `arm_1_link` → `arm_2_link` → ...), each at about 10 Hz. Sensor, LED and camera frames hang off `chassis_base_link` and `arm_2_link` as fixed links.

Deliverables

- D1: this report. D2: `ep_teleop_sdk.py`, `rm_setup.sh`, `scripts/`, raw CSV logs, ROS bag. D3: video link **[TODO: unlisted YouTube/cloud link, ≤45 s]**.